

8 - El mètode \$.getJSON()

JSON és l'acrònim de *JavaScript Object Notation*. És un format alternatiu d'enviament i recepció de dades, és a dir, reemplaça *XML* o l'enviament de text pla. Aquest format de dades és més lleuger que *XML*, veurem que fa el codi més senzill ja que utilitza el codi *JavaScript* com a model de dades.

Hem de repassar una mica com es defineixen els *arrays* i els objectes literals en *JavaScript*, ja que seran les estructures per a la transmissió de dades:

Els elements d'un *array* es tanquen entre claudàtors i els valors continguts van separats per coma:

```
var usuari = ['juan', 26];
```

Quan definim un *array* no indiquem a cada element de quin tipus es tracta, podem emmagatzemar cadenes (entre cometes), números, valors lògics (*true*, *false*), objectes i el valor *null*.

Per accedir als elements d'un *array* ho fem pel seu nom i entre claudàtors indiquem quins elements volem accedir:

```
alert(usuari[0]); //Imprimim el primer element de l'array  
alert(usuari[1]); //Imprimim el segon element de l'array
```

Vegem com definim els objectes literals a *JavaScript*:

```
var persona = {  
  'nom': 'juan',  
  'clau': 'xyz',  
  'edat': 26  
};
```

Els objectes literals es defineixen per mitjà de parells "nom": "valor". Tot objecte literal té un nom, a l'exemple li vaig anomenar *persona*. Un objecte literal conté una sèrie de propietats amb els seus valors respectius. Totes les propietats de l'objecte estan tancades entre claus. Cada propietat i valor se les separa per dos punts. I finalment cada propietat se les separa de les altres propietats per mitjà de la coma.

Per accedir a les propietats de l'objecte literal ho podem fer de dues maneres:

```
alert(persona.nom); //Imprimeix el valor de la propietat nom de l'objecte persona.
```

La segona forma és indicant la propietat entre claudàtors:

```
alert(persona['nom']);
```

Després es poden combinar objectes literals i *arrays*:

```
var persona = {  
  'nom': 'juan',  
  'edat': 22,  
  'estudis': ['primari', 'secundari']  
};
```

Com podem veure, es poden crear estructures de dades complexes combinant objectes literals i *arrays*. Per accedir als diferents elements tenim:

```
alert(persona.nom);  
alert(persona.estudis[0]);  
alert(persona.estudis[1]);
```

La sintaxi de *JSON* difereix lleument del que s'ha vist anteriorment:

```
{  
  'nom': 'juan',  
  'edat': 22,  
  'estudis': ['primari', 'secundari']  
}
```

Com podem veure a la sintaxi *JSON* no apareixen variables, sinó directament indiquem entre claus les propietats i els seus valors. També hem eliminat el punt i coma de la clau final. La resta és igual.

Ara vegem la diferència entre *JSON* i *XML* utilitzant aquest exemple:

XML:

```
<persona>
  <nom>juan</nom>
  <edat>22</edat>
  <estudis>
    <estudi>primari</estudi>
    <estudi>secundari</estudi>
  </estudis>
</persona>
```

I com vam veure a *JSON*:

```
{
  'nom': 'juan',
  'edat': 22,
  'estudis': ['primari', 'secundari']
}
```

Podem veure que la definició d'aquesta estructura és molt més directa. De tota manera, quan l'estructura que cal representar és molt complexa, *XML* continua sent l'opció principal.

Si hem de transmetre aquestes estructures per internet, la notació *JSON* és més lleugera.

Un altre avantatge és com recuperem les dades al navegador. En pràctiques anteriors hem vist com recuperar un fitxer *XML*, fent una crida *AJAX* mitjançant l'objecte *XMLHttpRequest*, recuperant l'*XML* a través de la variable *responseXML* i finalment processant la resposta per mitjà del *DOM*.

En canvi, amb *JSON* procedim a generar una variable en *JavaScript* que recreï l'objecte literal. Per provar i generar un objecte a partir d'una notació *JSON* farem el problema següent amb el mètode *\$.getJSON()* de *jQuery*:

La funció *\$.getJSON()* fa una petició de dades al servidor considerant que retorna la informació amb notació *JSON*. La sintaxi d'aquesta funció és:

```
$.getJSON(url, ["paràmetres"], ["funció resposta"])
```

El primer paràmetre és l'únic obligatori i indica la *url* a cridar. Opcionalment es poden enviar dades al servidor i finalment es defineix la funció resposta. La funció `$.getJSON()` procedeix a generar un objecte en *JavaScript* i nosaltres a la funció resposta el processem. Exemple:

```
function resposta(dades) {  
    alert(dades.nom)  
}
```

Problema: Confeccionar un lloc que permeti ingressar el document d'una persona i ens retorni el nom, el cognom i el lloc on ha de votar.